

Reproduce HOL(Head Of Line) blocking with default scheduler

argowf yaml: @spark-pi-workflow.yaml

1. simulate a large long-running spark job which consumes all(almost) resources:

```
argo submit spark-pi-workflow.yaml \  
  -p image=apache/spark:3.5.7 \  
  -p partitions=1000000 \  
  -p namespace=yunikorn-spark-submit \  
  -p executors=16
```

screenshot:

```

● → spark-submit git:(main) ✕ argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=1000000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=16
Name: spark-pi-wf-vg2lb
Namespace: yunikorn-spark-submit
ServiceAccount: unset (will run with the default ServiceAccount)
Status: Pending
Created: Sun Jan 04 17:02:02 +0800 (now)
Progress:
Parameters:
  image: apache/spark:3.5.7
  partitions: 1000000
  namespace: yunikorn-spark-submit
  executors: 16
  queue: root.default
  scheduler: default-scheduler
● → spark-submit git:(main) ✕ kgp
NAME                                READY   STATUS    RESTARTS   AGE
pythonpi-c0b3049b883e1f04-exec-1   1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-10  1/1     Running   0           84s
pythonpi-c0b3049b883e1f04-exec-11  1/1     Running   0           84s
pythonpi-c0b3049b883e1f04-exec-12  1/1     Running   0           83s
pythonpi-c0b3049b883e1f04-exec-13  0/1     Pending   0           83s
pythonpi-c0b3049b883e1f04-exec-14  0/1     Pending   0           83s
pythonpi-c0b3049b883e1f04-exec-15  0/1     Pending   0           83s
pythonpi-c0b3049b883e1f04-exec-16  0/1     Pending   0           82s
pythonpi-c0b3049b883e1f04-exec-2    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-3    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-4    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-5    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-6    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-7    1/1     Running   0           85s
pythonpi-c0b3049b883e1f04-exec-8    1/1     Running   0           84s
pythonpi-c0b3049b883e1f04-exec-9    1/1     Running   0           84s
spark-pi-930fdc9b883e03f9-driver    1/1     Running   0           96s
spark-pi-wf-vg2lb                    2/2     Running   0           98s

```

2. simulate a small short spark job, which we expect low latency:

```

argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=10000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=1

```

screenshot:

```

➔ spark-submit git:(main) argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=10000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=1
Name: spark-pi-wf-qkfx5
Namespace: yunikorn-spark-submit
ServiceAccount: unset (will run with the default ServiceAccount)
Status: Pending
Created: Sun Jan 04 17:03:54 +0800 (now)
Progress:
Parameters:
  image: apache/spark:3.5.7
  partitions: 10000
  namespace: yunikorn-spark-submit
  executors: 1
  queue: root.default
  scheduler: default-scheduler
➔ spark-submit git:(main) kubectl get pods --sort-by=.metadata.creationTimestamp
NAME                                READY   STATUS    RESTARTS   AGE
spark-pi-wf-vg2lb                   2/2     Running   0           4m47s
spark-pi-930fdc9b883e03f9-driver    1/1     Running   0           4m45s
pythonpi-c0b3049b883e1f04-exec-3    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-1    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-7    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-6    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-5    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-4    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-2    1/1     Running   0           4m34s
pythonpi-c0b3049b883e1f04-exec-9    1/1     Running   0           4m33s
pythonpi-c0b3049b883e1f04-exec-8    1/1     Running   0           4m33s
pythonpi-c0b3049b883e1f04-exec-11   1/1     Running   0           4m33s
pythonpi-c0b3049b883e1f04-exec-10   1/1     Running   0           4m33s
pythonpi-c0b3049b883e1f04-exec-15   0/1     Pending   0           4m32s
pythonpi-c0b3049b883e1f04-exec-14   0/1     Pending   0           4m32s
pythonpi-c0b3049b883e1f04-exec-13   0/1     Pending   0           4m32s
pythonpi-c0b3049b883e1f04-exec-12   1/1     Running   0           4m32s
pythonpi-c0b3049b883e1f04-exec-16   0/1     Pending   0           4m31s
spark-pi-wf-qkfx5                   2/2     Running   0           2m55s
spark-pi-3b4d599b884046c3-driver    0/1     Pending   0           108s

```

conclusion:

with default scheduler and limited resources, there will be HOL blocking, large long-running spark jobs consume all resources will block future jobs in the same ns.

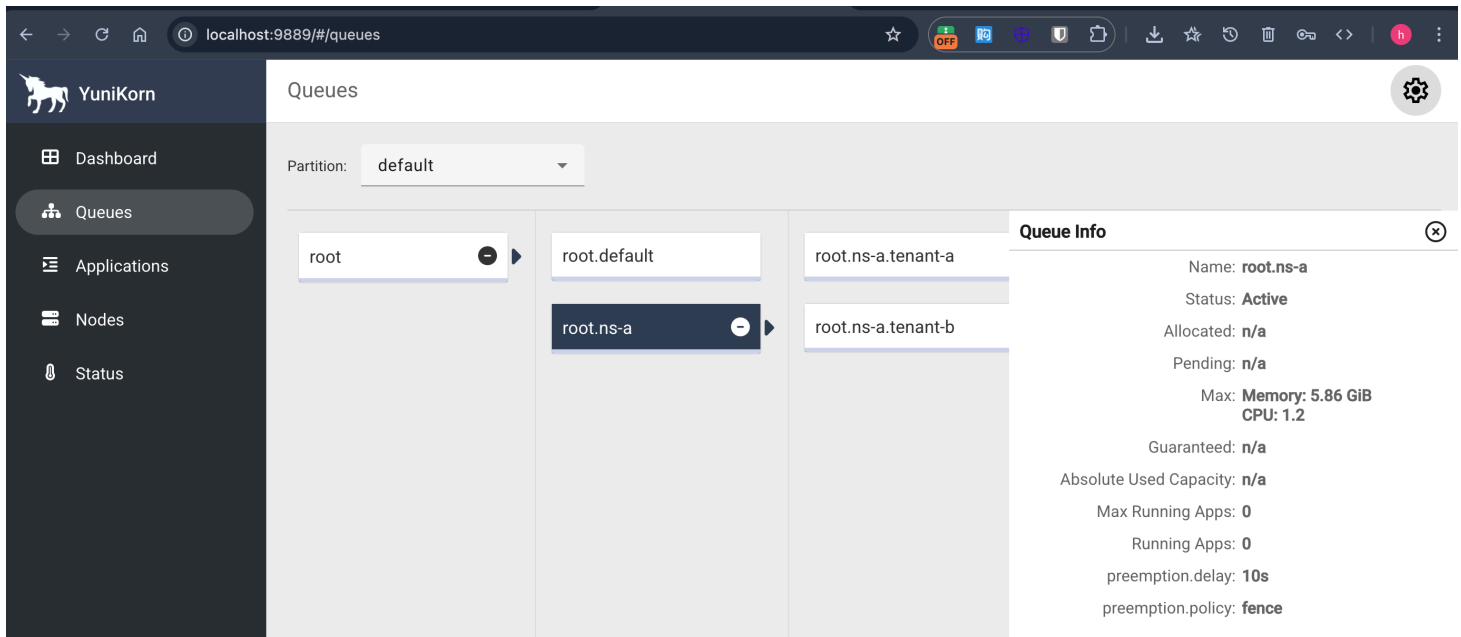
Resolve HOL blocking with yunikorn

1. clear all workloads:

```
→ spark-submit git:(main) argo delete --all
Workflow 'spark-pi-wf-qkfx5' deleted
Workflow 'spark-pi-wf-vg2lb' deleted
→ spark-submit git:(main) x kdel $(kubectl get pod -o name | grep spark-pi)
pod "spark-pi-3b4d599b884046c3-driver" deleted from yunikorn-spark-submit namespace
pod "spark-pi-930fdc9b883e03f9-driver" deleted from yunikorn-spark-submit namespace
spark-submit git:(main) x kgp
no resources found in yunikorn-spark-submit namespace.
→ spark-submit git:(main) x █
```

2. install yunikorn and config queues

parent queue:



The screenshot shows the YuniKorn web interface at localhost:9889/#/queues. The left sidebar contains navigation links: Dashboard, Queues (selected), Applications, Nodes, and Status. The main content area is titled 'Queues' and shows a tree structure of queues. The 'Partition' dropdown is set to 'default'. The queue tree shows 'root' as the parent, with children 'root.default' and 'root.ns-a'. 'root.ns-a' has children 'root.ns-a.tenant-a' and 'root.ns-a.tenant-b'. The 'root.ns-a' queue is selected, and its details are shown on the right in the 'Queue Info' panel.

Queue Info
Name: root.ns-a
Status: Active
Allocated: n/a
Pending: n/a
Max: Memory: 5.86 GiB CPU: 1.2
Guaranteed: n/a
Absolute Used Capacity: n/a
Max Running Apps: 0
Running Apps: 0
preemption.delay: 10s
preemption.policy: fence

child: root.ns-a.tenant-a

localhost:9889/#/queues

YuniKorn

Queues

Partition: default

root → root.default → root.ns-a.tenant-a

root.ns-a → root.ns-a.tenant-b

Queue Info

- Name: root.ns-a.tenant-a
- Status: Active
- Allocated: n/a
- Pending: n/a
- Max: Memory: 5.86 GiB
CPU: 1.2
- Guaranteed: Memory: 2.93 GiB
CPU: 600m
- Absolute Used Capacity: n/a
- Max Running Apps: 0
- Running Apps: 0
- preemption.delay: 10s
- preemption.policy: default
- priority.offset: 0
- Applications: [Link](#)

child: root.ns-a.tenant-b

localhost:9889/#/queues

YuniKorn

Queues

Partition: default

root → root.default → root.ns-a.tenant-a

root.ns-a → root.ns-a.tenant-b

Queue Info

- Name: root.ns-a.tenant-b
- Status: Active
- Allocated: n/a
- Pending: n/a
- Max: Memory: 5.86 GiB
CPU: 1.2
- Guaranteed: Memory: 2.93 GiB
CPU: 600m
- Absolute Used Capacity: n/a
- Max Running Apps: 0
- Running Apps: 0
- preemption.delay: 10s
- preemption.policy: default
- priority.offset: 0
- Applications: [Link](#)

3. simulate a large long-running spark job which consume all(almost) allocated resources:

specify yunikorn as the scheduler and the queue with low priority

```
argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=20000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=6 \
  -p queue=root.ns-a.tenant-a \
  -p scheduler=yunikorn
```

wait for some momemt will see:

```
→ spark-submit git:(main) x argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=20000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=6 \
  -p queue=root.ns-a.tenant-a \
  -p scheduler=yunikorn
Name: spark-pi-wf-4nxsq
Namespace: yunikorn-spark-submit
ServiceAccount: unset (will run with the default ServiceAccount)
Status: Pending
Created: Tue Jan 06 18:36:15 +0800 (now)
Progress:
Parameters:
  image: apache/spark:3.5.7
  partitions: 20000
  namespace: yunikorn-spark-submit
  executors: 6
  queue: root.ns-a.tenant-a
  scheduler: yunikorn
→ spark-submit git:(main) x kgp -l queue=root.ns-a.tenant-a
NAME                                READY    STATUS    RESTARTS    AGE
pythonpi-7d4ad09b92e12646-exec-1    1/1      Running   0            13s
pythonpi-7d4ad09b92e12646-exec-2    1/1      Running   0            13s
pythonpi-7d4ad09b92e12646-exec-3    1/1      Running   0            13s
pythonpi-7d4ad09b92e12646-exec-4    1/1      Running   0            13s
pythonpi-7d4ad09b92e12646-exec-5    1/1      Running   0            12s
pythonpi-7d4ad09b92e12646-exec-6    0/1      Pending   0            12s
spark-pi-f156f29b92e0ffd9-driver    1/1      Running   0            26s
→ spark-submit git:(main) x █
```

4. simulate a small short spark job, which we expect low latency:

specify yunikorn as the scheduler and the queue with normal priority, which is higher than the previous low priority queue, so can preempt pods from that queue

```
argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=10000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=2 \
  -p queue=root.ns-a.tenant-b \
  -p scheduler=yunikorn
```

wait for some momemt will see:

```
→ spark-submit git:(main) x argo submit spark-pi-workflow.yaml \
  -p image=apache/spark:3.5.7 \
  -p partitions=10000 \
  -p namespace=yunikorn-spark-submit \
  -p executors=2 \
  -p queue=root.ns-a.tenant-b \
  -p scheduler=yunikorn
Name: spark-pi-wf-mkxs5
Namespace: yunikorn-spark-submit
ServiceAccount: unset (will run with the default ServiceAccount)
Status: Pending
Created: Tue Jan 06 18:37:51 +0800 (now)
Progress:
Parameters:
  image: apache/spark:3.5.7
  partitions: 10000
  namespace: yunikorn-spark-submit
  executors: 2
  queue: root.ns-a.tenant-b
  scheduler: yunikorn
→ spark-submit git:(main) x kgp -l queue=root.ns-a.tenant-a
NAME                                READY    STATUS    RESTARTS    AGE
pythonpi-7d4ad09b92e12646-exec-1    1/1     Running    0            2m26s
pythonpi-7d4ad09b92e12646-exec-10    0/1     Pending    0            16s
pythonpi-7d4ad09b92e12646-exec-3     1/1     Running    0            2m26s
pythonpi-7d4ad09b92e12646-exec-7     0/1     Pending    0            51s
pythonpi-7d4ad09b92e12646-exec-8     0/1     Pending    0            51s
pythonpi-7d4ad09b92e12646-exec-9     0/1     Pending    0            32s
spark-pi-f156f29b92e0ffd9-driver     1/1     Running    0            2m39s
→ spark-submit git:(main) x kgp -l queue=root.ns-a.tenant-b
NAME                                READY    STATUS    RESTARTS    AGE
pythonpi-d06ea59b92e2bfa7-exec-1     1/1     Running    0            45s
pythonpi-d06ea59b92e2bfa7-exec-2     1/1     Running    0            45s
spark-pi-ab66ca9b92e27c37-driver     1/1     Running    0            64s
→ spark-submit git:(main) x █
```

1. some of the exec pods of the low priority job was preempted, and new ones created but in pending state.
2. the normal priority spark job was able to run, because it runs on a different queue with higher priority, and can preempt pods on the low priority queue when it consumes resources less than its guaranteed.

conclusion:

with yunikorn scheduler and prioritized queues, HOL blocking can be resolved, large long-running spark jobs running on low priority queues can be preempted by jobs running on higher priority queues. thus jobs with higher priority can get resource to execute.